Documentación del programa.

Tema: Gestión de Torneos Deportivos.

Integrantes: Denniston Tomás, Costa Agustin.

Descripción del programa

Desarrollamos un sistema en C para gestionar torneos deportivos desde la consola de comandos. El objetivo principal del programa es mantener la información guardada de forma permanente en un archivo de texto (torneos.txt), de modo que los datos no se borren al cerrar la aplicación.

A través de un menú interactivo, el usuario puede cargar nuevos torneos, consultar la lista completa de los registros guardados, filtrar torneos según el monto del premio o clasificar la información separando los torneos en dos archivos independientes (torneos_finalizados.txt y torneos_en_curso.txt) según su estado

Cómo organizamos los datos (`struct`)

Para representar cada torneo usamos una estructura (`struct`) que llamamos `torneo`. La armamos con los tipos de datos que pedía la consigna:

Campo	Tipo de Dato en C	Descripción / Propósito
nombre	char[56]	Nombre del torneo (hasta 55 caracteres).
deporte	char[56]	Deporte que se practica (hasta 55 caracteres).

`cantidadEquipos`	`int`	Cantidad de equipos participantes (número entero).
`premio`	`double`	Monto del premio en dinero (número real).
`terminado`	`int`	Estado del torneo (1 = Finalizado, 0 = En Curso).

Implementación Técnica de las Opciones del Menú

El control del sistema se estructuró mediante un bucle `do-while` y un `switch` en la función principal, delegando la ejecución a funciones auxiliares modularizadas:

- **Alta de registros (`agregarTorneo`):** Lee los datos de teclado aplicando validación de rangos y buffers. Abre el archivo principal en modo lectura y anexo (`"a"`) con `fopen` para preservar los registros previos e incluir el salto de línea correspondiente.
- **Lectura y Filtrado (`listarTorneos`):** Abre `torneos.txt` en modo lectura (`"r"`) y utiliza la función auxiliar `leerTorneo` para procesar el archivo secuencialmente mediante `fscanf`. Esta misma función gestiona tanto el listado completo como el filtrado por premio mínimo, evaluando la condición mediante una variable de control booleana.
- **Procesamiento y División (`dividirPorEstado`):** Abre el archivo de origen (`"r"`) y crea/sobrescribe dos archivos de salida (`"w"`): `torneos_finalizados.txt` y `torneos_en_curso.txt`. Recorre cada registro evaluando el campo booleano `terminado` (`1` o `0`) para direccionar la escritura a su respectivo archivo.

Decisiones de Diseño y Manejo de Archivos

- **Paso de parámetros y archivos (`FILE*`):** Toda la interacción con el sistema de archivos de C la centralizamos pasando las variables necesarias y los punteros de tipo `FILE*` directamente como argumentos a cada función (`fopen`, `fclose`, `fprintf`, `fscanf`). De esta forma evitamos el uso de variables globales y mantenemos el código ordenado por módulos.

- **Modos de apertura del archivo:**

  Para la **carga de torneos** (Opción 1), abrimos el archivo `torneos.txt` en modo anexar (`"a"`), lo que permite escribir los nuevos torneos al final sin borrar la información cargada anteriormente.

  Para las **lecturas y filtros** (Opciones 2 y 3), abrimos en modo lectura (`"r"`) y procesamos el archivo registro por registro.

  Para la **división por estado** (Opción 4), leemos el archivo principal y abrimos dos archivos nuevos en modo escritura (`"w"`): `torneos_finalizados.txt` y `torneos_en_curso.txt`, derivando cada torneo según el valor del campo booleano `terminado`.

- **Manejo de cadenas con espacios:** Dado que la función `fscanf` utiliza espacios para delimitar los campos del archivo de texto, acordamos guardar los textos compuestos utilizando guiones bajos (por ejemplo, `Copa_Libertadores`). Esto asegura que la lectura secuencial lea cada campo entero y no desajuste el formato.